

Note 2: Docker & GitHub Actions — Building and Pushing Images

Why Build Docker Images in CI?

Building Docker images locally and pushing them manually works for individuals but breaks down in teams. GitHub Actions allows every code change to automatically produce a versioned, tested Docker image and push it to a registry — ensuring the image is always built from the latest tested code and is immediately available for deployment.

Setting Up Docker Buildx

Docker Buildx is an extended build client that supports multi-platform builds and advanced caching. It is the recommended way to build images in GitHub Actions.

- name: Set up Docker Buildx

uses: docker/setup-buildx-action@v3

Authenticating with Docker Hub

Before pushing images, you must log in to the registry. Credentials are stored as GitHub repository secrets and injected into the workflow at runtime.

Adding Secrets to GitHub Repository

In your GitHub repository:

- Go to Settings → Secrets and variables → Actions
- Click New repository secret
- Add DOCKERHUB_USERNAME and DOCKERHUB_TOKEN (use an access token, not your password)

Login Step in Workflow

- name: Log in to Docker Hub

uses: docker/login-action@v3

with:

username: \${{ secrets.DOCKERHUB_USERNAME }}

password: \${{ secrets.DOCKERHUB_TOKEN }}

The docker/login-action handles authentication securely. Credentials are never printed in logs.

Building and Pushing to Docker Hub

name: Docker CI

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

docker:

runs-on: ubuntu-latest

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Set up Docker Buildx

uses: docker/setup-buildx-action@v3

- name: Log in to Docker Hub

if: github.event_name == 'push'

uses: docker/login-action@v3

with:

username: \${{ secrets.DOCKERHUB_USERNAME }}

password: \${{ secrets.DOCKERHUB_TOKEN }}

- name: Extract metadata for Docker

```
id: meta
uses: docker/metadata-action@v5
with:
  images: myusername/my-app
  tags: |
    type=ref,event=branch
    type=sha,prefix=sha-
    type=semver,pattern={{version}}
```

```
- name: Build and push Docker image
uses: docker/build-push-action@v5
with:
  context: .
  push: ${{ github.event_name == 'push' }}
  tags: ${{ steps.meta.outputs.tags }}
  labels: ${{ steps.meta.outputs.labels }}
  cache-from: type=gha
  cache-to: type=gha,mode=max
```

Explanation of Key Steps

docker/metadata-action Automatically generates appropriate image tags and labels based on the Git context:

- On a push to main — tags the image as myusername/my-app:main
- On a commit — tags as myusername/my-app:sha-a1b2c3d
- On a Git tag like v1.2.0 — tags as myusername/my-app:1.2.0

docker/build-push-action Builds the Docker image using the Dockerfile in the repository root and pushes it to the registry.

- context: . — uses the current directory as the Docker build context
- push: \${{ github.event_name == 'push' }} — only pushes on actual pushes to the branch, not on pull request runs (which would fail due to missing registry credentials)

- `cache-from/cache-to: type=gha` — uses GitHub Actions cache for Docker layer caching, significantly speeding up image builds when only application code changes
-

Pushing to GitHub Container Registry (GHCR)

GHCR is GitHub's own container registry, tightly integrated with GitHub repositories and permissions.

- name: Log in to GitHub Container Registry

uses: docker/login-action@v3

with:

registry: ghcr.io

username: `${{ github.actor }}`

password: `${{ secrets.GITHUB_TOKEN }}`

- name: Build and push to GHCR

uses: docker/build-push-action@v5

with:

context: .

push: true

tags: ghcr.io/`${{ github.repository_owner }}`/my-app:latest

Key differences from Docker Hub:

- Registry URL is ghcr.io
 - Username is `${{ github.actor }}` — the GitHub username that triggered the workflow
 - Password is `${{ secrets.GITHUB_TOKEN }}` — an automatically generated token available in every workflow; no manual secret creation needed
 - Image name follows the format `ghcr.io/<owner>/<image-name>:<tag>`
-

Combined Maven Build + Docker Push Workflow

This is the most realistic CI workflow for a Java Spring Boot project — it builds and tests with Maven, then packages into a Docker image and pushes to GHCR:

name: Java CI with Docker

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

build-and-push:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Set up Java 17

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

cache: 'maven'

- name: Build and test with Maven

run: mvn clean package

- name: Set up Docker Buildx

uses: docker/setup-buildx-action@v3

- name: Log in to GHCR

if: github.event_name == 'push'

uses: docker/login-action@v3

with:

registry: ghcr.io

username: \${{ github.actor }}

password: \${{ secrets.GITHUB_TOKEN }}

- name: Build and push Docker image

uses: docker/build-push-action@v5

with:

context: .

push: \${{ github.event_name == 'push' }}

tags: |

ghcr.io/\${{ github.repository_owner }}/my-app:latest

ghcr.io/\${{ github.repository_owner }}/my-app:\${{ github.sha }}

cache-from: type=gha

cache-to: type=gha,mode=max
